

STAP++ 二维平面弹性单元扩展报告

Q4 四边形单元与 T3 三角形单元

王冠泽

2023013092

有限元法大作业 I: 编程训练

2026 年 5 月 29 日

摘要

本文面向有限元法大作业 I 的“扩展 STAP++ 程序功能”要求，在原 STAP++ 框架中实现了两类二维平面弹性单元：Q4 四节点等参四边形单元和 T3 三节点常应变三角形单元。程序支持平面应力和平面应变两种材料模式，保持 STAP++ 原有输入输出风格，并完成材料读取、单元刚度装配、总体方程求解和单元应力输出。验证部分包括基础算例、分片试验、网格加密自收敛分析、原 Bar 单元回归测试以及非法输入测试。结果表明，Q4 与 T3 均能通过常应变分片试验，拉伸与剪切算例随网格加密呈现合理收敛趋势，新增功能未破坏原有 Bar 单元。

目录

1 引言	3
2 程序结构与输入格式	3
2.1 代码扩展位置	3
2.2 新增与修改文件清单	3
2.3 材料输入格式	4
2.4 单元输入格式	4
3 算法说明	5
3.1 平面弹性本构矩阵	5
3.2 Q4 四节点等参单元	5
3.3 T3 三节点常应变单元	5
4 实现方案	6
4.1 框架接入流程	6

目录	2
4.2 刚度矩阵与应力恢复实现	6
4.3 错误检查与稳定性处理	6
5 使用方法	7
6 程序验证与确认	7
6.1 基础算例与回归测试	7
6.2 分片试验	8
6.3 网格加密自收敛分析	8
6.4 异常输入测试	9
7 统一桥梁测试文件兼容性检查	9
8 任务分工与完成情况	10
9 版本控制与仓库说明	10
10 当前局限与可扩展方向	11
11 结论	11

1 引言

课程项目要求在 STAP++ 或 STAP90 基础上扩展有限元程序功能。个人作业至少需要增加一个单元，并给出单元基本原理、编程思路、输入数据格式、分片试验、收敛性分析和验证算例结果。为使作业内容更完整，本文没有只停留在单个 Q4 单元，而是在 Q4 的基础上进一步实现 T3 单元，使程序具备两类常用二维平面弹性单元。

本项目的主要功能如下：

- 实现 Q4 四节点等参四边形单元，采用 2×2 Gauss 积分；
- 实现 T3 三节点常应变三角形单元，采用解析常应变矩阵；
- 支持平面应力和平面应变两种本构矩阵；
- 输出每个二维单元的 σ_x 、 σ_y 和 τ_{xy} ；
- 提供 Q4 与 T3 的分片试验、网格加密验证和异常输入测试；
- 使用 Git 分支和 tag 存档原 Q4 版本，便于回滚。

2 程序结构与输入格式

2.1 代码扩展位置

本项目尽量保持 STAP++ 原结构不变，只按已有 Bar 单元风格增加新类和分支。核心修改包括：

- 新增 CQ4 和 CT3 单元类，分别位于 `src/h` 与 `src/cpp`；
- 在 `Material.h/.cpp` 中新增 `CQ4Material` 和 `CT3Material`；
- 在 `ElementGroup` 中根据单元类型分配对应的元素数组和材料数组；
- 在 `Outputter` 中增加 Q4/T3 的材料、单元连接关系和应力输出；
- 修正元素材料指针的所有权问题，避免析构时释放非拥有指针；
- 修正材料读取失败未向上传递的问题，使非法材料模式能正常报错。

2.2 新增与修改文件清单

为便于检查程序工作量，下面按类型列出本项目新增或重点修改的文件。除单元公式代码外，框架接入、材料输入、输出、自动化验证、算例数据和报告材料均已纳入仓库。

- 单元实现文件：`src/h/Q4.h`、`src/cpp/Q4.cpp`、`src/h/T3.h`、`src/cpp/T3.cpp`。其中 Q4 负责四节点等参映射、 2×2 Gauss 积分和中心点应力恢复；T3 负责三节点面积计算、常应变矩阵和单元常应力恢复。

- 材料与框架接入文件: `src/h/Material.h`, `src/cpp/Material.cpp`, `src/h/ElementGroup.h`, `src/cpp/ElementGroup.cpp`。这些文件新增 Q4/T3 材料类, 并使元素组能根据 `NPAR(1)` 正确分配 Q4 或 T3 对象。
- 输出文件: `src/h/Outputter.h`, `src/cpp/Outputter.cpp`。新增 Q4/T3 的材料参数、节点连接关系和 $\sigma_x, \sigma_y, \tau_{xy}$ 应力输出。
- 验证脚本: `others/tools/q4_generate_cases.py`, `others/tools/q4_parse_results.py`, `others/tools/t3_generate_cases.py`, `others/tools/t3_parse_results.py`。这些脚本负责自动生成算例并从输出文件中汇总位移和应力结果。
- 验证数据: `data/q4_generated/` 和 `data/t3_generated/` 存放分片试验、拉伸和剪切网格加密、非法输入等算例的输入输出文件。
- 报告与表格: `doc/q4_final_report.tex`, `doc/q4_final_report.pdf` 和 `doc/tables/`。其中 CSV 表格保存分片试验和网格加密验证的原始汇总数据。

2.3 材料输入格式

Q4 与 T3 采用相同的材料输入格式:

```
nset E nu thickness mode
```

其中 E 为弹性模量, ν 为泊松比, `thickness` 为单元厚度, `mode=1` 表示平面应力, `mode=2` 表示平面应变。若 `mode` 不是 1 或 2, 或厚度不为正, 程序会返回错误。

2.4 单元输入格式

二维单元均只使用节点的 x, y 坐标和 u_x, u_y 两个自由度。由于 STAP++ 节点类原本包含三个平移自由度, 所有二维算例都将 u_z 约束, 避免出现无刚度自由度。

Q4 单元组使用 `NPAR(1)=2`:

```
2 NUME NUMMAT
nset E nu thickness mode
element_id node1 node2 node3 node4 material_set
```

T3 单元组使用 `NPAR(1)=3`:

```
3 NUME NUMMAT
nset E nu thickness mode
element_id node1 node2 node3 material_set
```

两类单元均要求节点按逆时针方向输入。Q4 检查 Gauss 点的 $\det J$, T3 检查面积 A ; 若几何方向错误或单元退化, 程序报错停止。

3 算法说明

3.1 平面弹性本构矩阵

平面应力本构矩阵为

$$\mathbf{D} = \frac{E}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}. \quad (1)$$

平面应变本构矩阵为

$$\mathbf{D} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & (1-2\nu)/2 \end{bmatrix}. \quad (2)$$

单元应变向量和应力向量分别为

$$\boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_x & \varepsilon_y & \gamma_{xy} \end{bmatrix}^T, \quad \boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}. \quad (3)$$

3.2 Q4 四节点等参单元

Q4 单元采用自然坐标 (ξ, η) 中的双线性形函数:

$$N_1 = \frac{1}{4}(1-\xi)(1-\eta), \quad N_2 = \frac{1}{4}(1+\xi)(1-\eta), \quad (4)$$

$$N_3 = \frac{1}{4}(1+\xi)(1+\eta), \quad N_4 = \frac{1}{4}(1-\xi)(1+\eta). \quad (5)$$

等参映射由 $x = \sum N_i x_i$ 、 $y = \sum N_i y_i$ 给出。程序在每个 Gauss 点计算 Jacobi 矩阵和应变-位移矩阵 $\mathbf{B}$ ，并用 2×2 Gauss 积分计算刚度矩阵:

$$\mathbf{K}_e = \sum_{g=1}^4 \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det(\mathbf{J}_g) t. \quad (6)$$

应力输出取单元中心点 $(\xi, \eta) = (0, 0)$ 的结果。

3.3 T3 三节点常应变单元

T3 单元面积为

$$A = \frac{1}{2}[(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)]. \quad (7)$$

定义

$$b_i = y_j - y_k, \quad c_i = x_k - x_j, \quad (8)$$

其中 (i, j, k) 循环取 $(1, 2, 3)$ 、 $(2, 3, 1)$ 、 $(3, 1, 2)$ 。T3 的应变-位移矩阵为常量:

$$\mathbf{B} = \frac{1}{2A} \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}. \quad (9)$$

因此单元刚度矩阵为

$$\mathbf{K}_e = t\mathbf{A}\mathbf{B}^T\mathbf{D}\mathbf{B}. \quad (10)$$

T3 的应变和应力在单元内为常量，这使其实现简单、稳定，适合作为第二个扩展单元。

4 实现方案

程序实现遵循“最小改动、保持原框架”的原则。STAP++ 原程序已经具有节点、材料、元素组、总体装配、skyline 矩阵和 LDLT 求解器等基础结构，因此本项目没有重写求解器，而是将 Q4/T3 做成与 Bar 单元并列的派生单元。

4.1 框架接入流程

新增单元进入程序的流程如下：

1. 输入文件中读取 NPAR(1)，确定元素组类型。Bar、Q4、T3 分别对应不同枚举值。
2. ElementGroup 根据元素类型分配正确的材料数组和元素数组。
3. 材料对象读取 E, ν, t, mode ，并判断平面应力或平面应变。
4. 单元对象读取节点编号和材料组号，保存节点指针和材料指针。
5. 单元生成位置矩阵，只取每个节点的 u_x, u_y 两个自由度参与二维平面问题。
6. 总体装配流程调用每个单元的 ElementStiffness，求解后再调用 ElementStress 输出单元应力。

4.2 刚度矩阵与应力恢复实现

Q4 与 T3 的共同点是都先建立 $\mathbf{B}$ 矩阵和材料矩阵 $\mathbf{D}$ ，再计算 $\mathbf{K}_e$ 。不同点在于：Q4 的 $\mathbf{B}$ 随 Gauss 点变化，需要通过 Jacobi 矩阵将自然坐标导数转换到物理坐标；T3 的 $\mathbf{B}$ 在单元内为常量，可以直接由节点坐标和面积求得。

应力恢复时，程序先由位置矩阵从总体位移向量中取出单元节点位移，再计算

$$\boldsymbol{\varepsilon}_e = \mathbf{B}\mathbf{u}_e, \quad \boldsymbol{\sigma}_e = \mathbf{D}\boldsymbol{\varepsilon}_e. \quad (11)$$

Q4 输出中心点应力，T3 输出单元常应力。这样输出格式统一，便于后续比较 Q4/T3 的平均应力和网格收敛趋势。

4.3 错误检查与稳定性处理

本项目增加了几类必要检查：

- 材料 mode 必须为 1 或 2，避免本构矩阵未定义；

- 厚度必须大于零，避免刚度矩阵无物理意义；
- Q4 在 Gauss 点检查 $\det J > 0$ ，发现反向或严重退化四边形；
- T3 检查 $A > 0$ ，发现顺时针节点顺序或零面积单元；
- 修正材料读取错误未向上传递的问题，使错误输入能终止程序。

Q4 和 T3 均复用 STAP++ 的 skyline 总刚度矩阵和 LDLT 求解器，不改变全局装配和求解流程。这样做的优点是风险小，原 Bar 单元行为保持不变；需要注意的是，二维单元必须显式约束 u_z ，否则会出现没有刚度贡献的自由度。

5 使用方法

在 Windows PowerShell 下可按如下方式构建和运行：

```
cmake -S src -B build
cmake --build build --config Debug
.\build\Debug\stap++.exe data\q4-single.dat
.\build\Debug\stap++.exe data\t3-generated\t3-single.dat
```

自动化验证脚本位于 others/tools/。生成和解析 T3 验证数据的命令为：

```
python others/tools/t3_generate_cases.py
python others/tools/t3_parse_results.py
```

Q4 对应脚本为 q4_generate_cases.py 和 q4_parse_results.py。

6 程序验证与确认

6.1 基础算例与回归测试

本项目运行了原 Bar 算例、已有 Q4 算例和新增 T3 算例。Bar 回归用于确认原功能未被破坏；Q4/T3 基础算例用于检查读入、装配、求解、应力输出是否形成闭环。运行结果显示，右侧受拉节点的水平位移为正，拉伸算例的平均 σ_x 与外力方向一致，剪切算例的平均 τ_{xy} 与施加载荷方向一致。主要验证算例包括：

- data/bar-6.dat：原 Bar 单元回归测试，确认原程序功能未被破坏。
- data/q4-single.dat、data/q4-plane-strain.dat、data/q4-two-element.dat、data/q4-shear.dat：分别验证 Q4 的基本拉伸、平面应变材料、小网格装配和剪切应力输出。
- data/q4_generated/q4_patch_2x2.dat：Q4 常应变分片试验。
- data/t3_generated/t3-single.dat、data/t3_generated/t3-two-element.dat：分别验证 T3 单元级拉伸和两个三角形拼成正方形的装配效果。

- data/t3_generated/t3_patch_2x2.dat: T3 常应变分片试验。
- Q4/T3 非法输入算例: 验证非法材料模式、反向节点顺序、非正 Jacobi 行列式和非正面积检查。

6.2 分片试验

分片试验采用 2×2 网格和线性位移场

$$u = ax, \quad v = by, \quad (12)$$

其中 $a = 1.0 \times 10^{-3}$, $b = -5.0 \times 10^{-4}$ 。为避免修改 STAP++ 的非零位移边界输入格式, 验证脚本在程序外部组装刚度矩阵, 并用 $\mathbf{f} = \mathbf{K}\mathbf{u}$ 反算等效节点力。该方法可以检验单元能否准确再现常应变状态。

表 1: Q4 与 T3 分片试验结果

单元	最大 u_x 误差	最大 u_y 误差	σ_x 平均误差	σ_y 平均误差
Q4	0.0	0.0	1.54×10^{-4}	4.62×10^{-5}
T3	0.0	0.0	1.54×10^{-4}	4.62×10^{-5}

理论应力为 $\sigma_x = 1.961538 \times 10^2$ 、 $\sigma_y = -4.615385 \times 10^1$ 、 $\tau_{xy} = 0$ 。表中的应力误差主要来自输出文件保留五位科学计数法导致的截断; 位移误差为零, 说明 Q4 与 T3 均通过常应变分片试验。

6.3 网格加密自收敛分析

网格加密算例包括拉伸和剪切两类。左边界固定, 右边界施加等效集中节点力。由于本阶段不再进行 ABAQUS 对比, 收敛性以 8×8 网格结果作为参考值进行自收敛比较。

表 2: 拉伸算例网格加密结果

单元	网格	最大 u_x	平均 σ_x	相对差异
Q4	1×1	4.62010×10^{-3}	1.00000×10^3	2.14×10^{-2}
Q4	2×2	4.68107×10^{-3}	1.00000×10^3	8.44×10^{-3}
Q4	4×4	4.71071×10^{-3}	9.99999×10^2	2.16×10^{-3}
Q4	8×8	4.72091×10^{-3}	1.00000×10^3	0
T3	1×1	4.85511×10^{-3}	9.99999×10^2	2.28×10^{-2}
T3	2×2	4.86852×10^{-3}	1.00000×10^3	2.56×10^{-2}
T3	4×4	4.79445×10^{-3}	1.00000×10^3	9.98×10^{-3}
T3	8×8	4.74706×10^{-3}	1.00000×10^3	0

表 3: 剪切算例网格加密结果

单元	网格	最大 u_y	平均 τ_{xy}	相对差异
Q4	1×1	1.10053×10^{-2}	5.00000×10^2	3.51×10^{-1}
Q4	2×2	1.39833×10^{-2}	5.00000×10^2	1.75×10^{-1}
Q4	4×4	1.60605×10^{-2}	5.00000×10^2	5.27×10^{-2}
Q4	8×8	1.69541×10^{-2}	5.00000×10^2	0
T3	1×1	8.58918×10^{-3}	5.00000×10^2	4.73×10^{-1}
T3	2×2	1.14415×10^{-2}	5.00000×10^2	2.99×10^{-1}
T3	4×4	1.45211×10^{-2}	5.00000×10^2	1.10×10^{-1}
T3	8×8	1.63122×10^{-2}	5.00000×10^2	0

结果表明, Q4 和 T3 在拉伸与剪切算例中均随网格加密逐渐接近参考值。T3 在粗网格下与 Q4 有差异, 这是因为 T3 为常应变三角形单元, 同一正方形区域需要两个三角形近似; 加密后其结果同样趋于稳定。

从误差趋势看, Q4 在拉伸算例中相对差异从 2.14×10^{-2} 下降到 2.16×10^{-3} , 剪切算例中从 3.51×10^{-1} 下降到 5.27×10^{-2} 。T3 在剪切问题中粗网格误差更大, 但从 1×1 到 4×4 也呈明显下降趋势。这与单元理论相符: Q4 的双线性位移模式对矩形网格较有效, T3 的常应变模式需要更细网格才能描述弯曲或剪切变形梯度。

本报告采用自收敛验证而非外部软件对比, 原因是课程要求允许使用解析解或构造人工解进行收敛率分析。本项目中的分片试验属于构造人工解, 网格加密算例则用于检查程序装配和单元刚度在更大网格下的稳定性。

6.4 异常输入测试

异常测试用于确认程序能够发现明显错误输入:

- Q4 非法材料模式: q4-invalid-mode.dat;
- Q4 反向节点顺序导致 $\det J \leq 0$: q4-invalid-jacobian.dat;
- T3 非法材料模式: t3-invalid-mode.dat;
- T3 反向节点顺序导致 $A \leq 0$: t3-invalid-area.dat。

上述算例均按预期报错停止, 说明材料模式检查和单元几何检查有效。

7 统一桥梁测试文件兼容性检查

课程统一测试文件 课程设计测试文件.zip 中给出了四个 Abaqus .inp 桥梁模型和一份模型说明。说明文件指出, 该桥梁模型用于测试程序能力和效率, 桥墩使用实体单元, 桥面使用板单元, 桥面下方加固结构使用梁单元, 上方吊索使用杆单元。四个模型规模如表 4 所示。

表 4: 统一桥梁测试模型规模统计

模型	节点数	S4R	C3D8R	B31	T3D2
Bridge-1	4163	400	1760	704	20
Bridge-2	37185	2500	26500	1350	20
Bridge-3	259567	10000	220000	2700	20
Bridge-4	1910327	40000	1760000	5420	20

本项目对 `Bridge-1.inp` 进行了直接兼容性检查。当前 STAP++ 程序的输入接口要求 STAP 风格的 `.dat` 文件；将 Abaqus `.inp` 文件直接传入程序时，程序返回错误：

```
*** Error *** Invalid file extension: inp
```

该结果符合预期，说明统一桥梁模型不是当前程序可以直接读取的输入格式。

进一步从单元类型看，完整桥梁模型需要 S4R 板单元、C3D8R 实体单元、B31 梁单元和 T3D2 杆单元；而本项目实现的是 STAP++ 框架下的 Q4 和 T3 二维平面弹性单元，并保留原 Bar 单元。两者面向的问题维度和输入格式不同。因此，本项目不声称能够直接求解四个 Bridge 全桥模型。

这次兼容性检查的意义在于明确程序能力边界：当前成果已经满足个人作业“至少增加一个单元”的基本要求，并额外实现了 Q4+T3 两类二维平面单元；若后续要支持课程统一桥梁模型，则需要继续完成两方面工作：一是实现或接入梁、板、实体等三维结构单元；二是编写 Abaqus `.inp` 到 STAP++ 输入数据结构的转换器。

8 任务分工与完成情况

本项目为个人大作业，未采用团队分工。本人独立完成以下内容：阅读课程项目要求和 STAP++ 原程序；设计并实现 Q4 与 T3 二维平面弹性单元；编写材料输入、单元读入、刚度矩阵、应力恢复和异常检查代码；生成并运行 Bar 回归、Q4 验证和 T3 验证算例；整理 CSV 数据表；撰写并编译最终报告；使用 Git 进行版本控制和 GitHub 提交。

9 版本控制与仓库说明

项目使用 Git 进行版本控制，仓库来源和提交位置如下：

- 提交仓库：<https://github.com/Albert041118/STAPpp>；
- 原始上游程序：<https://github.com/xzhang66/STAPpp>；
- 本地上游远端：upstream 指向课程指定 STAP++ 仓库；
- 主线分支：main；

仓库根目录按课程要求保留 `src`、`make`、`data`、`doc` 和 `others`。其他课程资料、原始参考文档和辅助脚本统一放入 `others`。

为便于从 T3 扩展回滚到原 Q4 最终版本，在新增 T3 前已创建备份 tag:

```
backup/q4-final-before-t3
```

T3 开发分成两个主要提交：第一个提交实现 T3 单元代码和框架接入，第二个提交加入 T3 验证算例、CSV 表格和报告更新。最终修改已合并并推送到 GitHub 主分支。

10 当前局限与可扩展方向

当前实现已经满足二维平面弹性单元扩展和验证要求，但仍有进一步扩展空间：

- 目前 Q4 输出中心点应力，T3 输出常应力；若要做更精细后处理，可进一步输出 Gauss 点应力或节点平均应力。
- 当前 STAP++ 输入格式不支持非零位移边界条件，因此分片试验通过外部脚本反算等效节点力实现；若以后扩展输入格式，可直接施加非零位移边界。
- 当前未实现 TecPlot/ParaView 云图输出；后续可将节点位移和单元应力导出为 VTK 或 TecPlot 格式，用于绘制变形图和应力云图。
- 当前未加入带宽优化；对于更大规模网格，可进一步实现 RCM 节点重排序以提高 skyline 存储效率。

这些内容不影响本项目对 Q4/T3 单元基本原理、程序实现、输入格式、分片试验和网格加密验证的完整展示。

11 结论

本文完成了 STAP++ 的二维平面弹性功能扩展。Q4 单元体现了等参映射、Jacobi 检查和数值积分；T3 单元体现了常应变三角形公式、面积检查和与 Q4 的对比验证。两类单元均支持平面应力和平面应变，均通过常应变分片试验，并在拉伸和剪切网格加密算例中表现出合理的自收敛趋势。相比只实现一个 Q4 单元，Q4+T3 的组合更完整地展示了二维有限元单元实现、验证和程序扩展能力。同时，本文对课程统一 Bridge 测试文件进行了兼容性检查，明确了当前程序尚不能直接求解 Abaqus 全桥模型，并给出了后续扩展方向。

参考资料

1. 张雄，有限元法基础课程大作业说明，2026。
2. X. Zhang, T. Wang, Y. Liu, *Computational Dynamics*, Tsinghua University Press.
3. STAP++ 源程序：<https://github.com/xzhang66/STAPpp>。